CYBER SECURITY INTERNSHIP REPORT
(BEGINNER LEVEL TASKS)

Under the Guidance of ShadowFox

Submitted By:

Ashish Agrawal

Course: BCA

University: UPES, Dehradun

Organization: ShadowFox

Duration: 1st March – 31st March

Date of Submission: 29 March 2026

TABLE OF CONTENTS

Contents

TABLE OF FIGURES 2

ACKNOWLEDGEMENT 3

INTRODUCTION 4

INFORMATION 5

OBJECTIVES 6

TASK EXECUTION 7

Task 1: Port Scanning 7

Task 2: Directory Brute Forcing 10

Task 3: Network Traffic Analysis using Wireshark 14

REFERENCES 19

RESOURCES USED 20

TABLE OF FIGURES

Figure 1: Nmap scan showing open ports and services 8
Figure 2: Gobuster output showing discovered directories on the target web application 11
Figure 3: Login page of target web application 14
Figure 4: Captured network packets in Wireshark 15
Figure 5: Filtered HTTP traffic showing login request 15
Figure 6: HTTP stream showing transmitted credentials 16

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to ShadowFox for providing me with the opportunity to undertake this Cyber Security Internship. This internship allowed me to gain practical exposure to essential cybersecurity concepts and tools such as port scanning, directory enumeration, and network traffic analysis.

I would also like to thank the mentors and the ShadowFox team for their continuous guidance and support throughout the internship. Their structured approach and practical tasks helped me understand real-world cybersecurity scenarios more effectively.

This experience has strengthened my foundational knowledge in cybersecurity and has motivated me to further develop my skills in this field.

INTRODUCTION

Cybersecurity is an essential domain in modern information technology that focuses on protecting systems, networks, and data from unauthorized access, attacks, and damage. With the increasing reliance on digital platforms and online services, ensuring the security of information has become a critical priority for organizations and individuals alike.

The growing number of cyber threats, including data breaches, malware attacks, and unauthorized intrusions, highlights the need for effective security measures and skilled professionals in this field. Cybersecurity plays a vital role in safeguarding sensitive data and maintaining the integrity and availability of systems.

This report documents the tasks and activities performed during the Cyber Security Internship at ShadowFox. It provides a detailed overview of the beginner-level tasks assigned, including port scanning, directory brute forcing, and network traffic analysis. The report also explains the tools used, methodologies followed, and the results obtained during the execution of these tasks, offering insights into practical cybersecurity techniques and real-world applications.

INFORMATION

The tasks assigned during this internship were performed in a controlled virtual environment to ensure safe and effective execution of cybersecurity techniques. The primary operating system used was Kali Linux, which provides a wide range of tools specifically designed for penetration testing and security analysis.

The system was set up using a virtual machine environment, allowing isolated testing without affecting the host system. Various tools such as Nmap, Gobuster, and Wireshark were utilized to perform tasks including port scanning, directory enumeration, and network traffic analysis.

This setup enabled a practical understanding of real-world cybersecurity scenarios and ensured that all activities were conducted in a secure and ethical manner.

Operating System: Kali Linux

Tools Used: Nmap, Gobuster, Wireshark

Environment: Virtual Machine (VirtualBox)

Target Website: http://testasp.vulnweb.com

OBJECTIVES

- To understand the fundamentals of cybersecurity and ethical hacking
- To perform port scanning and identify open services on a target system
- To conduct directory brute forcing to discover hidden paths
- To analyze network traffic using Wireshark and identify transmitted data
- To gain practical experience with cybersecurity tools and techniques

TASK EXECUTION

Task 1: Port Scanning

Objective:

The objective of this task is to identify open ports and the services running on the target web application http://testasp.vulnweb.com in order to understand potential attack surfaces.

Tool Used:

Nmap (Network Mapper)

Command Used:

nmap -p- -sV testasp.vulnweb.com -oN scan.txt

Command Explanation:

Nmap is a network scanning tool used to discover hosts and services on a network. The "-p-" flag is used to scan all 65, 535 TCP ports, ensuring that no open ports are missed. The "-sV" flag is used to detect the version of services running on open ports. This helps in identifying exact software and versions, which can be used for vulnerability assessment and "-oN scan.txt" saves the scan results in a normal output file for further analysis.

Explanation:

Nmap is a widely used network scanning tool that helps in identifying open ports and services on a target system. By scanning all ports and detecting service versions, it provides valuable insights into the exposed services of a web application. These services can potentially be analyzed further for vulnerabilities, making port scanning an essential step in penetration testing.

```
┌──(ashish2704㉿Ashish-Kali)-[~/Desktop/shadowfox]
└─$ nmap -p- -sV testasp.vulnweb.com -oN scan.txt
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-21 03:28 EDT
Nmap scan report for testasp.vulnweb.com (44.238.29.244)
Host is up (0.33s latency).
rDNS record for 44.238.29.244: ec2-44-238-29-244.us-west-2.compute.amazonaws.com
Not shown: 65357 filtered tcp ports (no-response), 177 filtered tcp ports (admin-prohibited)
PORT   STATE SERVICE VERSION
80/tcp open  http    Microsoft IIS httpd 8.5
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1074.72 seconds
```

Figure 1: Nmap scan showing open ports and services

Result / Analysis:

The Nmap scan results indicate that the target host is active and reachable. A full port scan was performed, and it was observed that most of the ports were filtered, meaning they are either blocked by a firewall or not responding.

The scan identified port 80 as open, which is commonly used for HTTP web services. The service running on this port is Microsoft IIS httpd version 8.5, indicating that the web server is hosted on a Windows-based system.

The presence of filtered ports suggests that network-level security controls such as firewalls are in place, limiting visibility of other services. However, the open HTTP port confirms that the web application is accessible and can be further tested for vulnerabilities such as misconfigurations, outdated software, or insecure communication.

Conclusion:

The port scanning process successfully identified the active service running on the target system. The open HTTP port represents a potential entry point for further security assessment. The identified web server and its version can be analyzed for known vulnerabilities, making this step crucial in the penetration testing process.

Security Insight:

Open ports increase the attack surface of a system. If the services running on these ports are not properly secured or updated, attackers can exploit known vulnerabilities.

Therefore, it is important to regularly monitor open ports and restrict unnecessary services.

Attack Name:

Port Scanning (Information Gathering)

Severity:

Low (CVSS Score: 3.1)

Justification:

Port scanning is a reconnaissance technique used to identify open ports and running services on a target system. Although it does not directly exploit any vulnerability, it provides critical information that can be used by attackers to identify potential entry points and plan further attacks.

Impact:

The attack itself does not impact confidentiality, integrity, or availability. However, it exposes information about active services and open ports, which can assist attackers in targeting specific vulnerabilities or misconfigurations in the system.

Steps to reproduce:

1. Open terminal in Kali Linux.
2. Run the command:

   nmap -p- -sV testasp.vulnweb.com/ -oN scan.txt
3. Wait for the scan to complete.
4. Analyze the output to identify open ports and services.

Mitigation:

- Close unnecessary open ports on the server.
- Use firewalls to restrict access to essential services only.
- Implement intrusion detection system (IDS) to monitor scanning.
- Regularly update and patch services to prevent exploitation.

Task 2: Directory Brute Forcing

Objective:

The objective of this task is to discover hidden directories and files on the target web application in order to identify potential entry points and sensitive resources.

Tool Used:

Gobuster

Command Used:

gobuster dir -u http://testasp.vulnweb.com -w /usr/share/wordlists/dirb/common.txt -o dir_bruteforce

Command Explanation:

The Gobuster command is used to perform directory enumeration on the target web application. The “dir” mode specifies directory brute forcing. The “-u” flag defines the target URL, while the “-w” flag specifies the wordlist used for brute forcing. The “-o” option saves the output results to a file for further analysis.

Explanation:

Gobuster is a tool used to discover hidden directories and files on a web server by performing brute force attacks using a predefined wordlist. By sending multiple HTTP requests with different directory names, it helps in identifying accessible paths that may not be directly visible through normal browsing. This is an important step in penetration testing as hidden directories may contain sensitive data or vulnerable endpoints.

```
┌──(ashish2704㉿Ashish-Kali)-[~/Desktop/shadowfox]
└─$ gobuster dir -u http://testasp.vulnweb.com -w /usr/share/wordlists/dirb/common.txt -o dir_bruteforce
===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://testasp.vulnweb.com
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.6
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/_vti_cnf             (Status: 301) [Size: 159] [--> http://testasp.vulnweb.com/_vti_cnf/]
/aspnet_client        (Status: 301) [Size: 164] [--> http://testasp.vulnweb.com/aspnet_client/]
/avatars              (Status: 301) [Size: 158] [--> http://testasp.vulnweb.com/avatars/]
/cgi-bin              (Status: 301) [Size: 158] [--> http://testasp.vulnweb.com/cgi-bin/]
/cgi-bin/             (Status: 403) [Size: 1233]
Progress: 1819 / 4615 (39.41%)[ERROR] context deadline exceeded (Client.Timeout or context cancellation while reading body)
[ERROR] Get "http://testasp.vulnweb.com/full": context deadline exceeded (Client.Timeout exceeded while awaiting headers)
/html                 (Status: 301) [Size: 155] [--> http://testasp.vulnweb.com/html/]
/HTML                 (Status: 301) [Size: 155] [--> http://testasp.vulnweb.com/HTML/]
/images               (Status: 301) [Size: 157] [--> http://testasp.vulnweb.com/images/]
/Images               (Status: 301) [Size: 157] [--> http://testasp.vulnweb.com/Images/]
/jscripts             (Status: 301) [Size: 159] [--> http://testasp.vulnweb.com/jscripts/]
Progress: 3216 / 4615 (69.69%)[ERROR] Get "http://testasp.vulnweb.com/psp": context deadline exceeded (Client.Timeout exceeded while awaiting headers)
/robots.txt           (Status: 200) [Size: 13]
/t                    (Status: 301) [Size: 152] [--> http://testasp.vulnweb.com/t/]
/T                    (Status: 301) [Size: 152] [--> http://testasp.vulnweb.com/T/]
/templates            (Status: 301) [Size: 160] [--> http://testasp.vulnweb.com/templates/]
Progress: 4587 / 4615 (99.39%)[ERROR] Get "http://testasp.vulnweb.com/wstat": context deadline exceeded (Client.Timeout exceeded while awaiting headers)
Progress: 4614 / 4615 (99.98%)
===============================================================
Finished
===============================================================
```

Figure 2: Gobuster output showing discovered directories on the target web application

Result / Analysis:

The Gobuster scan successfully identified multiple directories on the target web application. Several directories such as "/images", "/templates", "/avatars", and "/aspnet_client" were discovered with HTTP status code 301, indicating redirection to accessible resources.

The directory "/robots.txt" returned a status code 200, indicating that the file exists and is accessible. This file can provide useful information about restricted or hidden paths on the website.

Some directories such as "/cgi-bin/" returned a 403 status code, indicating that access to these directories is forbidden. This suggests that access controls are in place, but the existence of these directories still reveals part of the application's structure.

Additionally, several requests resulted in timeout errors, which may indicate server-side filtering, rate limiting, or network latency issues during the scan.

Overall, the enumeration process revealed important directories that can be further analyzed for vulnerabilities such as insecure file access, misconfigurations, or sensitive data exposure.

Conclusion:

The directory brute forcing process successfully identified multiple accessible and restricted directories on the target web application. These findings highlight potential areas for further security testing and emphasize the importance of securing hidden resources and restricting unauthorized access.

Security Insight:

Hidden directories can expose sensitive information if not properly secured. Attackers can use directory enumeration techniques to discover unprotected endpoints, configuration files, or backup data. Proper access controls and directory restrictions are essential to minimize such risks.

Attack Name:

Directory Enumeration (Brute Force Attack)

Severity:

Medium (CVSS Score: 5.3)

Justification:

Directory enumeration allows an attacker to discover hidden directories and files without authentication. If sensitive endpoints are exposed, it can lead to unauthorized access or information disclosure, resulting in a partial impact on confidentiality.

Impact:

The attack can reveal hidden directories, configuration files, backup files, or administrative panels. If these resources are not properly secured, they may allow attackers to access sensitive data or identify further attack vectors, increasing the risk of system compromise.

Steps to reproduce:

1. Open terminal in Kali Linux.
2. Run the command:

   gobuster dir -u http://testasp.vulnweb.com -w /usr/share/wordlists/dirb/common.txt
3. Wait for the scan to complete.
4. Analyze the discovered directories and status code in the output.

Mitigation:

- Disable directory listing on the web server.
- Restrict access to sensitive directories using proper authentication.
- Remove unnecessary or unused files from the server.
- Implement a Web Application Firewall (WAF) to detect and block brute force attempts.

Task 3: Network Traffic Analysis using Wireshark

Objective:

The objective of this task is to capture and analyze network traffic to identify sensitive information such as login credentials transmitted over the network.

Tool Used:

Wireshark

Procedure:

Network traffic was captured using Wireshark while performing a login operation on the target web application. The capture was initiated on the active network interface (eth0).

A display filter “http” was applied to isolate HTTP traffic. The captured packets were analyzed to identify HTTP POST requests containing login data.

The “Follow HTTP Stream” feature was used to reconstruct the communication between the client and the server, allowing detailed inspection of transmitted data.

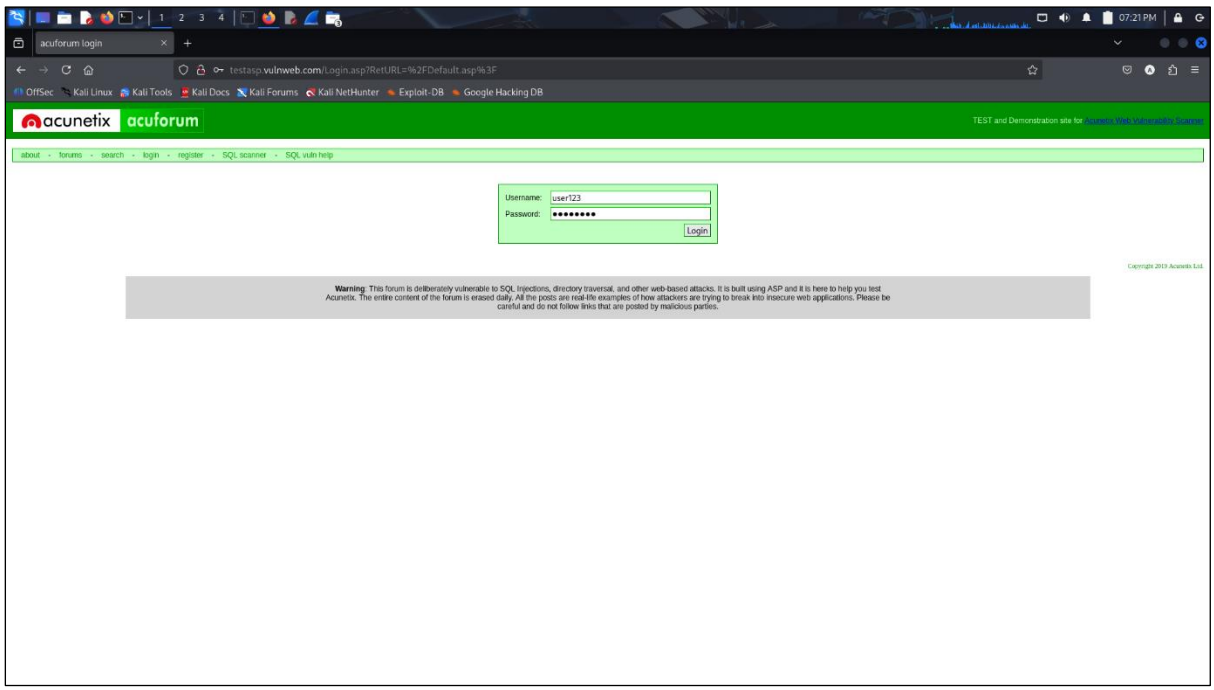

Figure 3: Login page of target web application

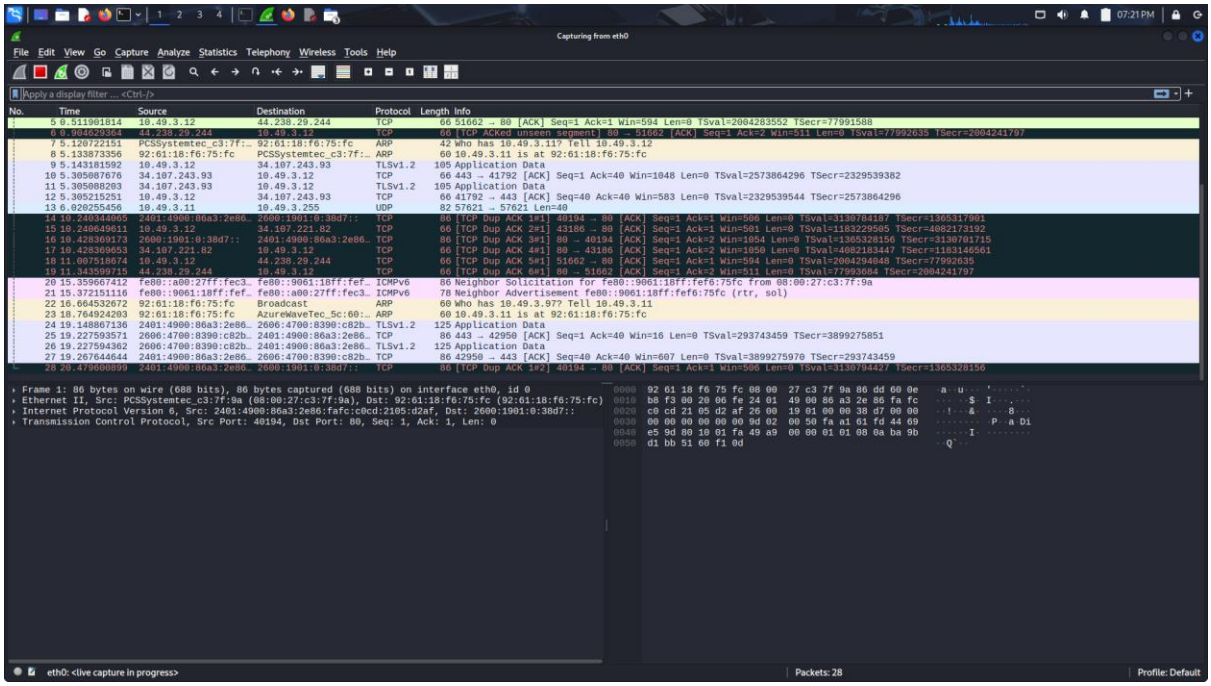

Figure 4: Captured network packets in Wireshark

Explanation:

Wireshark is a network protocol analyzer that captures and inspects packets transmitted over a network. It enables deep analysis of protocols such as HTTP, TCP, and others.

When data is transmitted over HTTP, it is not encrypted, making it possible to view sensitive information such as usernames and passwords directly in the packet data. By analyzing HTTP POST requests, credentials submitted through login forms can be extracted.

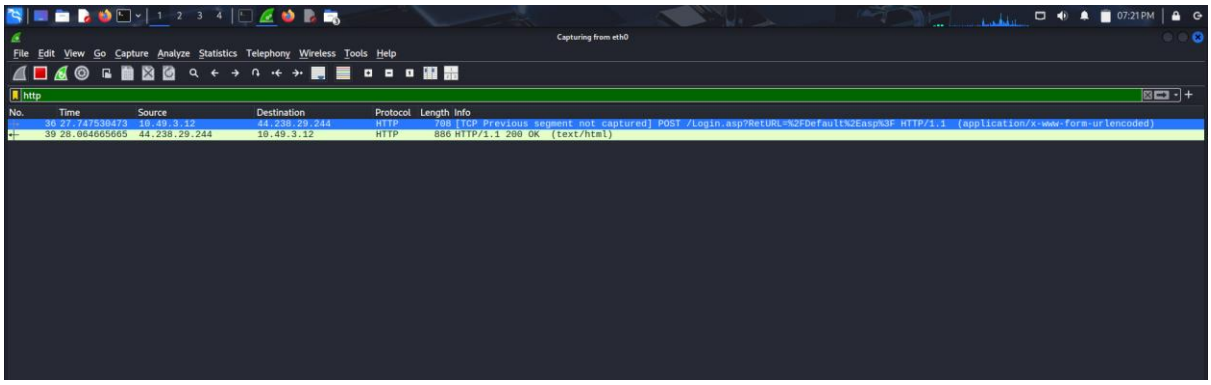

Figure 5: Filtered HTTP traffic showing login request

Result / Analysis:

The captured network traffic revealed that login credentials were transmitted in plain text over HTTP. By analyzing the HTTP POST request in the packet stream, the username and password were clearly visible.

The extracted data included:

Username: user123

Password: pass1234

This indicates that the application does not use encryption for transmitting sensitive information. Since HTTP does not provide any security, the credentials can be intercepted by anyone monitoring the network traffic.

The use of the "Follow HTTP Stream" feature allowed reconstruction of the full communication, making it easy to identify sensitive data within the request body.

This represents a serious security vulnerability, as attackers can capture login credentials and gain unauthorized access to user accounts.

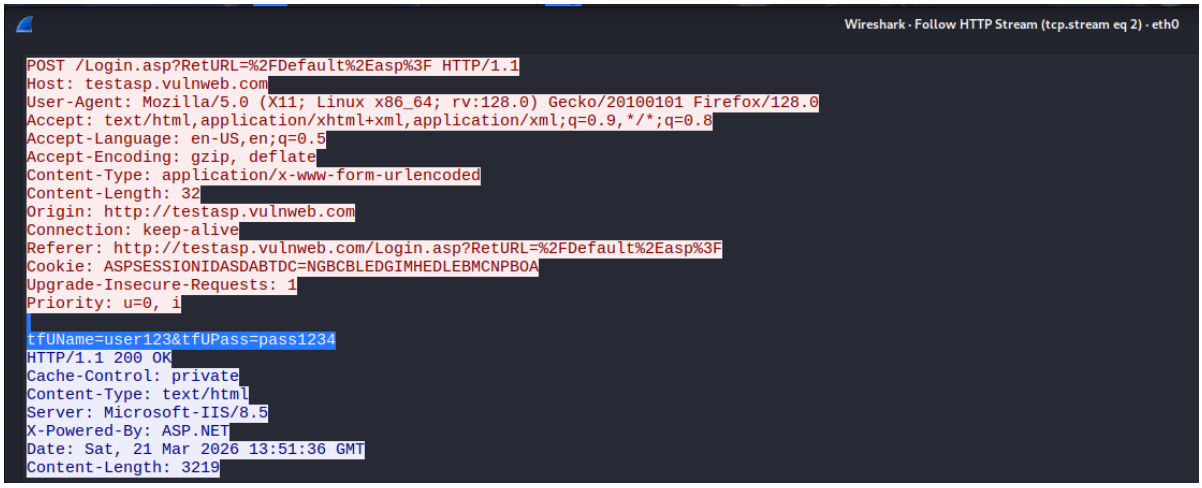

Figure 6: HTTP stream showing transmitted credentials

Conclusion:

The network traffic analysis successfully demonstrated that sensitive information such as login credentials can be intercepted when transmitted over an unencrypted protocol. This highlights the importance of secure communication mechanisms in web applications.

Security Insight:

Transmitting credentials over HTTP exposes users to packet sniffing attacks. Attackers can intercept network traffic and extract confidential information without needing direct access to the system, making encryption essential for secure communication.

Attack Name:

Packet Sniffing (Credential Interception)

Severity:

High (CVSS Score: 8.2)

Justification:

This vulnerability allows direct exposure of sensitive user credentials, which can lead to immediate account compromise.

Impact:

Sensitive user credentials such as usernames and passwords can be intercepted by attackers monitoring network traffic. This can result in unauthorized access, account takeover, and potential data breaches.

Steps to reproduce:

1. Open Wireshark and start capturing traffic on the active network interface.
2. Visit the target website and navigate to the login page.
3. Enter valid credentials and submit the login form.
4. Apply the filter:
   http
5. Locate the HTTP POST request.
6. Use “Follow HTTP Stream” to view transmitted data.
7. Observe the username and password in plain text.

Mitigation:

- Use HTTPS instead of HTTP to encrypt communication.
- Implement SSL/TLS certificates on the server.
- Avoid transmitting sensitive data in plain text.
- Use secure authentication mechanisms and encryption protocols.
- Implement network security measures to prevent packet sniffing attacks.

REFERENCES

-

1. Nmap Official Documentation – https://nmap.org/docs.html
   (Used for understanding port scanning and service detection)
2. Gobuster GitHub Repository – https://github.com/OJ/gobuster
   (Used for understanding port scanning and service detection)
3. Wireshark Official Documentation – https://www.wireshark.org/docs/
   (Used for packet analysis and traffic inspection)
4. Acunetix Test Website – http://testasp.vulnweb.com
   (Used to understand vulnerabilities related to directory exposure and insecure communication)
5. OWASP Web Security Testing Guide – https://owasp.org/www-project-web-security-testing-guide/
   (Used to understand risks of sensitive data exposure)
6. Kali Linux Tools Documentation – https://www.kali.org/tools/
   (Target platform used for performing security testing tasks)

RESOURCES USED

1. Kali Linux

   Used as the primary operating system for performing all cybersecurity tasks.
2. Nmap

   Used for port scanning to identify open ports and running services on the target system.
3. Gobuster

   Used for directory brute forcing to discover hidden directories on the web application.
4. Wireshark

   Used for capturing and analyzing network traffic to identify sensitive data transmission.
5. Worldlists (dirb/common.txt)

   Used for performing directory enumeration using Gobuster.
6. Web Browser

   Used to access the target website and perform login operations for traffic analysis.
7. Acunetix Test Website (http://testasp.vulnweb.com)

   Used as the target platform for performing penetration testing tasks.